

How (and Why) to Properly Compute the Entropies of Phonological Grammars.

Joe Bratsvedal Collins

cleverbymistake@outlook.com

Entropy is a critical concept upon which most theoretical linguistics implicitly depends – though through a cruel twist of history most linguists are unaware of this. Rather, entropy is often misunderstood in both phonology and linguistics more generally. Most commonly, entropy is mistaken for a single mathematical quantity, rather a more general concept that must be quantified differently depending on the specific context.

The general concept of entropy will not be dealt with here in full – it is a topic that has been covered by many other authors (e.g. Goldsmith 2000; Futrell & Hahn 2022; etc) – suffice it to say that entropy means *information* in the symbolic sense of “distinguishing between possibilities”, whether that be distinguishing between different features, phones, words, sentences, etc. And it is precisely the ability to distinguish possibilities that all formal phonological analyses concern themselves with. What divides formal phonology from information theory is largely the fact that formal phonology has thus far tended to consider information in purely non-quantitative terms. Therefore, this short paper describes the analytical methods for correctly computing several entropies, in relation to phonological grammars, in the hope that these concepts and methods can become more widespread.

This paper also implicitly restricts the discussion to the entropies of surface forms. However, it should be fairly clear that these methods also apply equally to underlying forms, and by extension the entropy change in the derivation of surface forms from underlying forms – which is surely the ultimate point of interest for any such phonological study.

Three Important Types of Entropy

There are three quantities of entropy of inherent interest to phonologists: the first is the maximum entropy of the alphabet itself, i.e. how much information *could* be transmitted with a given phone inventory in the absence of any grammar. This can be properly referred to as the Hartley function of the alphabet, or the capacity of the alphabet.

The second quantity is the capacity of the grammar itself. That is, what is the maximum amount of information the *could* be transmitted given both the phone inventory and the possible output strings producible by the grammar. This is computable analytically from a formal model of the grammar, and can be more properly regarded as the topological entropy.

The third quantity is the actual amount of information transmitted on average, given both the grammar and the frequencies of the individual phones. This is computed both from a model of the grammar and an empirical study of the phone frequencies of the language in question. This can be more properly referred to as measure-theoretic entropy, or Kolmogorov-Sinai entropy.

It is also worth noting what we are not interested in: namely, what is sometimes called the “Shannon entropy”, or entropy-rate of the alphabet. This is the quantity achieved by plugging the frequencies of all phones in the language into Shannon’s equation (Shannon 1948). In practice this is of little use to phonologists since it quantifies the information transmitted by a sender who knows the grammar, to a receiver who does not – which is not a natural condition of interest to phonologists.

Of course, Shannon's key insight was to show that it is possible to approximate the measure-theoretic entropy of an information source by computing the Shannon entropy of n-grams produced by the source. However, if we have already analytically computed the Topological entropy of the grammar, then it is a relatively simple task to analytically compute the measure-theoretic entropy as well, making n-gram approximation seem needlessly inaccurate and data-intensive in our case.

Computing the Hartley Function

The Hartley function is trivial to compute and is simply the log of the number of phones in the alphabet. The base of the logarithm is not important, provided it is kept consistent. However, it is relatively common to use base 2, since gives all quantities in bits, which are intuitively interpretable in a variety of ways (e.g. number of binary features/coin flips/etc.).

Despite its simplicity, the Hartley function of the alphabet is important since it is an absolute upper bound on information-rate for the alphabet. In the (purely theoretical) scenario where the grammar imposed no restrictions on surface forms and all phones occurred in equal probability, then the measure-theoretic entropy would simply equal the Hartley function. This point matters because it allows us to conceive of phonological rules as "trading off" a certain amount of information-rate for some (presumed) benefit in robustness, either in the speech signal or the underlying representations. This is an old idea, even in phonology¹, but computing the Hartley function is the first step in actually quantifying any such trade-off.

Computing the Capacity of the Grammar (Topological Entropy)

The Topological entropy is computed analytically from a model of the grammar². However it is considerably more intensive than the Hartley function. First, the output of the grammar must be represented as the state-transition graph of a finite-state machine, then one computes the largest eigenvalue of the adjacency matrix of said graph – the log of this eigenvalue is exactly the topological entropy of the grammar.

The difficulty here lies in the construction of the finite-state model of the grammar. It is well-known that all phonological grammars are sub-regular (e.g. Heinz & Idsardi 2013), and thus this problem is always solvable. However, to achieve an accurate answer it is essential both that the graph be a complete representation of the entire phonological grammar, and also that the graph be a *minimal* projection of that grammar, i.e. the FSM should require the minimal possible number of states to produce the correct output. In general, we should not expect formal phonological analyses to meet these criteria, partly because it is common practice for phonologists to analyse only parts of a grammar (e.g. one or two interacting processes), and partly because phonologists are often concerned with various aesthetic concerns which pay no heed to minimality. Thus, while a formal analysis of a grammar is a necessary starting point for computing topological entropy, there is nonetheless a certain amount of work associated with actual computation.

The topological entropy is useful because it tells us the informational capacity of the grammar, i.e. the maximum information-rate that could be conveyed by the alphabet, given the restrictions of the grammar. Therefore, the difference between the Hartley function and the topological entropy can be thought of as the *redundancy* introduced by the grammar. That is, a phonological rule makes certain output forms ungrammatical (i.e. impossible), meaning that the average (possible) word length increases, with the trade-off that it introduces a mechanism for error detection. Concretely, if a language introduces final devoicing then the lexicon loses e.g. [tag] as a possible word, but any listener who hears [tag] can identify a perceptual error without needing to check the lexicon (or

1 See e.g. Heinz & Idsardi (2013) discussion of Kaye (1989).

2 See Lind & Marcus (2021) and Marcus, Roth & Siegel (2001) for more technical details.

alternatively knows to expect more of the word, etc.). Exploiting redundancy in this way is exactly how error correcting codes work in artificial machines. And we can easily intuit the trade-off here: too many such rules and there are not enough distinct words left in the lexicon, and too few and the language becomes more prone to transmission errors. The difference between the Hartley function and the topological entropy is simply the natural quantification of this.

Information Rate (Measure Theoretic Entropy)

Given the graph model used to compute the Topological entropy, computing the measure-theoretic entropy is relatively simple: by assigning frequencies to the state transitions of the graph, the FSM becomes a Markov chain, whose entropy can be computed in the usual manner (Shannon 1948).

Of course, correctly computing the transition probabilities entails parsing some database using the FSM model. However, given the existence of large, annotated phonological corpora in several languages, this is by no means impossible.

The measure theoretic entropy is a quantification of how much information is actually transmitted *on average* given the frequencies of the phones in the language. The upper bound of the measure theoretic entropy is the topological entropy. The intuition behind this quantity is that, if a language only uses a small number of the phones in its inventory frequently, while ignoring others, then it is transmitting information less efficiently than if it used all phones equally often.

One interesting consequence of the distinction between topological and measure theoretic entropy, is the implication that certain phonological rules could be more (or less) optimal given the statistics of the language. This leads to a more mathematically meaningful formulation of the reasoning behind the idea of Functional Load (Hockett 1966, etc), as discussed under the final section of this paper.

Why Entropy Matters – Phonology as Constrained Coding

With the basic mathematical quantities in place, it's worth considering why they should be interesting to phonologists, which is an argument that follows from viewing phonology as a natural case of a problem frequently encountered in artificial computing systems. From the perspective of coding theory, phonological forms are a type of constrained code – a code where certain strings are prohibited. It is useful therefore to consider in detail an example of an artificial constrained code, before considering how this applies to natural phonology.

The textbook example of a constrained code is the Modified Frequency Modulation code (MFM) which is a particular solution to the (1,3)RLL³ constraint (Lind & Marcus 2021:2.5). The MFM is a binary code that was used on certain magnetic storage devices (e.g. floppy disks). These devices stored binary codes using the magnetic polarity on small cells placed on ferromagnetic track. As we shall see, the physical properties of these ferromagnetic tracks place limitations on the types of binary codes that can be reliably stored on them.

Since magnetic polarity is also a kind of binary representation (i.e. + and -), one might assume that the ones and zeroes of the code are simply mapped directly onto the two magnetic polarities (e.g. 1=+; 0=-). However, for reasons that will become clear, this is not the case. Instead, a binary “1” is realised as a *flip* in the polarity of a given cell, relative to its predecessor. So, the binary code “0010100” would be realized as follows (reading from left to right):

3 Where RLL stands for “run-length limited”.

Binary Code:	0	0	1	0	1	0	0
Magnetic	+	+	-	-	+	+	+
Track:							

Realizing the ones as “flips”, rather than a one-to-one mapping, has the advantage of producing longer magnetic cells of a given polarity, which in turn reduces the chances of error from interfering magnetic fields. However, this method nonetheless leaves two potential sources of error: Firstly, consecutive ones can cause the magnetic fields of adjacent cells to cancel out, thereby making the track harder to read. Secondly, because the ones are used as markers to synchronize the clocks of the storage device and the read/write device, too many consecutive zeros can allow the clocks to desynchronize, which also results in errors. To solve both these problems then, what is needed is a constrained code which allows neither adjacent ones nor too many zeros consecutively. This is where the (1,3)RLL constraint derives its name: the “1” and the “3” refer respectively to the minimum and maximum number of zeros between two ones.

Of course, while it is easy enough to define a code which contains no adjacent ones and at most three adjacent zeros, the critical engineering problem lies in creating an encoder/decoder which can output (1,3)RLL code for any given input. In practice, the MFM encoder/decoder works by “epenthesising” additional bits, in order to break up adjacent ones and/or runs of zeroes.

However, unlike many phonological patterns, the MFM encoder does not epenthesize *only* when epenthesis is licensed by context. That is, it does not epenthesize only when the input code contains adjacent ones or too many zeros. Rather, the encoder epenthesizes a bit between every adjacent pair of bits in the input code; epenthesizing a 1 if the input pair are both zeros, and a zero otherwise.

There are at least two reasons for preferring this form of encoding over a purely context-driven epenthesis. Firstly, it greatly simplifies the problem of decoding, since there is no need to determine which bits are epenthetic – the decoder can simply assume that every other bit is epenthetic and apply the encoding algorithm in reverse. Secondly, it simplifies the computational problem of encoding/decoding, by allowing for a smaller detection window. That is, the encoder/decoder does not need to “count” to find triplets of zeros, instead it needs only to consider adjacent pairs of input bits.

Thus, the MFM code is a particular solution to the (1,3)RLL constraint which balances the trade-off between error prevention, the length of the code, and the complexity of the encoder/decoder. The code prevents inter-symbol interference by epenthesizing additional bits into the output code, which naturally results in a longer code. In fact, by epenthesizing additional bits between *every* input pair, the coding scheme effectively doubles length of the code. This is, of course, undesirable in a storage device, but carries with it the advantage of a much simpler encoder/decoder.

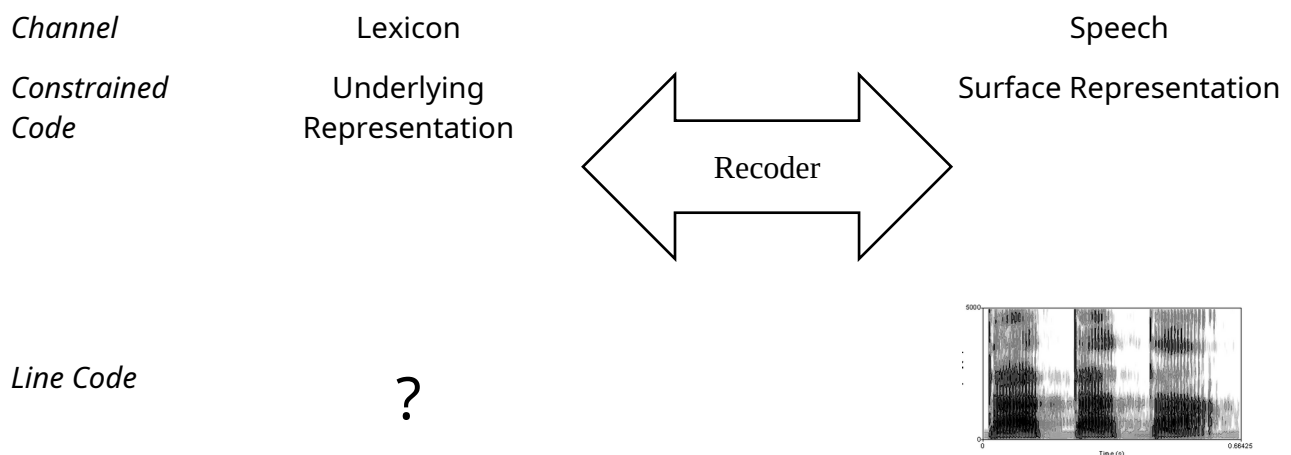
These are the sorts of trade-offs engineers must make when finding optimal coding schemes. And, as we shall see, these are also the trade-offs that natural languages must make, albeit with a different set of physical constraints.

Aspects of the MFM code should seem familiar to the phonologist. For example, inserting an epenthetic 1 between two 0s could seem like an example of the Obligatory Contour Principle. Yet, beyond the striking-yet-superficial similarities between the MFM and phonological rules, there is a sense in which phonology and MFM coding can be understood as different solutions to the same fundamental set of design problems: i.e. they are both *constrained codes*. Fully appreciating the consequences of this for phonologists, however, requires us first to examine in more detail the differences between the MFM encoder and the architecture of phonological grammars.

First, it is useful to import some terminological distinctions from coding theory. As already noted, MFM coding was employed on magnetic storage devices (e.g. floppy disks). We can say then that floppy disk is the *channel* of the MFM code. A *channel* is typically understood as information transmission between a sender and receiver (Shanon 1948) – this is also true in the case of data storage, however the data is being transmitted in time rather than space, and the sender and receiver will (likely) be the same encoder/decoder device. However, the basic mathematical treatment of a *channel* remains the same regardless of whether one is transmitting through space or time.

The physical encoding of information in a given channel can be referred to as its *line code*. In the case of the MFM code, the *line code* refers to the mapping between binary ones and zeros and the polarity of the magnetic cells on the disk. As we saw with the MFM example, the *line code* is not at all trivial since it is precisely the *line code* of a given *channel* that gives rise to the physical constraints which a given *constrained code* must satisfy.

In the case of the phonology then, we could posit a minimal interpretation of the phonological faculty as two channels, linked by a single encoder/decoder. Each of these channels corresponds to a different line code, which provides its own set of constraints:



The reason phonology is complex and interesting is precisely because these two codes appear to have different sets of physical constraints – surface forms are constrained by phonetics, while underlying forms are constrained by whatever principles govern encoding in the brain. Meanwhile, the recoder itself must also be limited by physical or general computational properties. Thus, any given phonological grammar represents a biological and sociological engineering solution to satisfying all three of these factors (underlying constraints, surface constraints, computational recoding constraints), while also being beholden to the path dependence of diachronic sound change.

At some level the previous paragraph might seem like a banal restatement of the standard view of phonology. Crucially however, the mathematical tools of coding theory also entail a neat conceptual distinction between a physical constraint and a code that satisfies that constraint – e.g. the difference between the (1,3)RLL constraint and the MFM code. From this perspective it becomes clear that many long-running arguments in phonology, e.g. phonetic reductionism, are based on simple misunderstanding. For example, while it may be quite true that many observable phonological processes are clearly satisfying some phonetic constraint, this observation itself does not “explain” the phonological process – the hard problem lies in understanding how the phonology efficiently encodes information *given* that constraint (as well as all the others). The exact same reasoning applies to constraints imposed by memory (etc.). Thus, any attempt to reduce phonology to something else, be it phonetics, memory, or computation, is fundamentally missing the interesting

problem. At its heart, phonology has always been concerned with the information theoretic problem of representing and recoding in a way that attempts to satisfy all of these constraints, because it is only the differing sets of constraints that make recoding necessary in the first place.

The coding theory perspective also reveals a fundamental flaw with traditional formal phonology, namely that it lacks a way of quantifying how good or bad any given representation would actually be at satisfying these various constraints. For example, phonologists can point to a given process and say “this *seems* phonetically unnatural, therefore it must be caused by something else.” – but the consequences of such an observation in practice amount to little more than vague speculations. If one takes seriously the idea of phonology-as-coding however, then it becomes clear that it is precisely the need to encode information *efficiently* that makes constraint satisfaction a non-trivial problem⁴. Thus, quantifying the entropies of the posited codes is an essential step in understanding how any given code attempts to satisfy any given constraint.

An Empirical Prediction

Finally, this paper concludes with a simple prediction, broken into two hypotheses, which are entirely testable given our current knowledge and technical capability. There are:

Hypothesis #1:

Diachronic sound changes should be driven towards minimizing the difference between the topological and measure-theoretic entropies of the surface forms.

Hypothesis #2:

When hypothesis #1 does not appear to hold, it is evidence for some other constraints influencing optimal language design (e.g. memory, computation, etc.)

The hypotheses are driven by the observation they would hold in a perfectly optimal system. That is, we suppose that the grammar wishes to impose some rules, which create redundancy allowing for a kind of soft error correction. Thus, the lower the topological entropy, the more (theoretical) redundancy that can be used for error correction. However, the grammar also wishes to maximize the information rate. Thus, the higher the measure-theoretic entropy the more efficient the speech signal. Since the topological entropy is an upper bound on measure theoretic entropy, the perfectly optimal case would be where these two values are equal. Any difference between the topological and measure theoretic entropies is “wasted space” that is neither being used to transmit information, nor being used for redundancy. Thus we can posit that diachronic shifts are periodic attempts by speakers/learners to reoptimize their speech signal, given small continual changes in the frequencies of individual phones (driven by external factors).

Anyone familiar with the Functional Load literature (Hockett 1966, etc) will notice the similarity here. However, a flaw with Functional Load is that it is typically formulated in terms of a single quantity (e.g. Shannon entropy), rather than an interaction between different quantities. For example, Hockett simply speculates that diachronic sound changes are triggered by some need to meet an arbitrary amount of redundancy. But there is no obvious clue as to where such an arbitrary target comes from. If however, we conceive of the optimization problem as an attempt to reduce the “wasted space” between topological and measure-theoretic entropy, then there is no need to posit an

⁴ For example, suppose that underlying and surface forms can be infinitely long, and the recoder can map any underlying form to any surface form. Constraint satisfaction is now a trivial problem.

arbitrary redundancy-target for the phonology – the entire process is motivated by uncontroversial reality if information transmission given conflicting constraints.

Moreover, closer analysis of such an optimization process shows a kind of symmetry breaking, where relatively small differences in phone frequency can suddenly make a given diachronic change preferable. This suggests a mechanism for the bifurcation of different dialects (and languages) once speaker groups become separated. And since any given diachronic change will alter both the topological entropy, *and* the frequencies of the phones, this process seems well suited to produce the never-ending cycles of change that appear to characterize language diachrony.

References

Futrell, R., & Hahn, M. (2022). Information theory as a bridge between language function and language form. *Frontiers in Communication*, 7, 657725.

Goldsmith, J. (2000). On information theory, entropy, and phonology in the 20th century.

Heinz, J., & Idsardi, W. (2013). What complexity differences reveal about domains in language. *Topics in cognitive science*, 5(1), 111-131.

Hockett, C. F. (1966). THE QUANTIFICATION OF FUNCTIONAL LOAD--A LINGUISTIC PROBLEM.

Kaye, J. (1989). Phonology: A cognitive view. Hillsdale, NJ: L. Erlbaum Associates.

Lind, D., & Marcus, B. (2021). *An introduction to symbolic dynamics and coding*. Cambridge university press.

Marcus, B. H., Roth, R. M., & Siegel, P. H. (2001). An introduction to coding for constrained systems. *Lecture notes*.

Shannon, C. E. (1948). A mathematical theory of communication. *The Bell system technical journal*, 27(3), 379-423.